

计算机组成原理第一次实验报告

实验名称：加法器改进

学号：2410668

姓名：吕昊远

班次：张金、董前琨

一、实验目的

1. 熟悉 LS-CPU-EXB-002 实验箱和软件平台；
2. 掌握利用该实验箱各项功能开发组成原理和体系结构实验的方法；
3. 理解并掌握加法器的设计原理；
4. 复习 Verilog 语法，熟练运用电路设计语言；
5. 深入理解加法器原理，学习 Verilog 改进设计思路。

二、实验内容

1. 阅读实验箱相关文档，熟悉硬件平台，掌握利用显示屏观察特定信号的方法。学习软件平台和设计流程。
2. 熟悉计算机中加法器的原理，并依此绘制设计逻辑图与结构图。
3. 根据设计的实验方案，使用 verilog 编写相应代码，并进行仿真验证，得到正确的波形图。
4. 生成位流文件，将其导入实验箱烧录，验证实验箱各项功能。
5. 完成基础的实验流程后，依据加法器原理，完成对于加法器的改进，改进要求如下：
 - 将原始的两个 32 位数的加法器，改进成四个 32 位数的加法器；
 - 修改 testbench 模块，通过波形仿真验证改进加法器的正确性；
 - 修改 display 模块和约束文件，通过上实验箱验证加法器正确性。

三、实验原理图

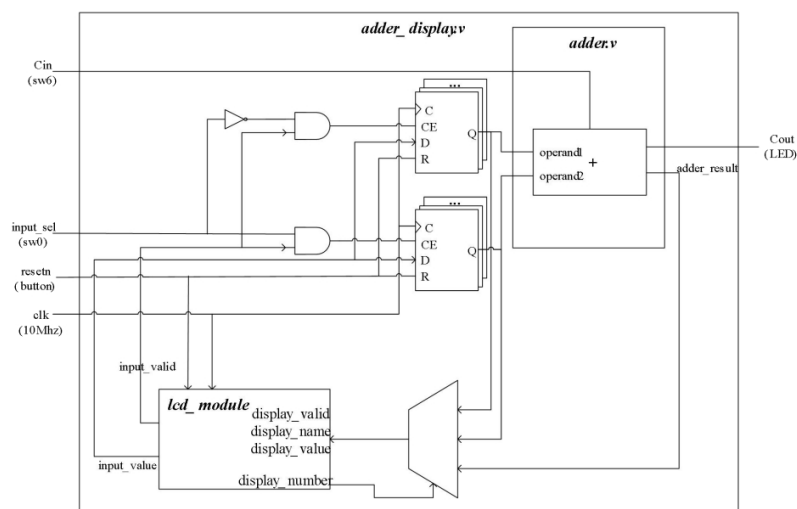


图 1: 实验原理图: 二元加法器

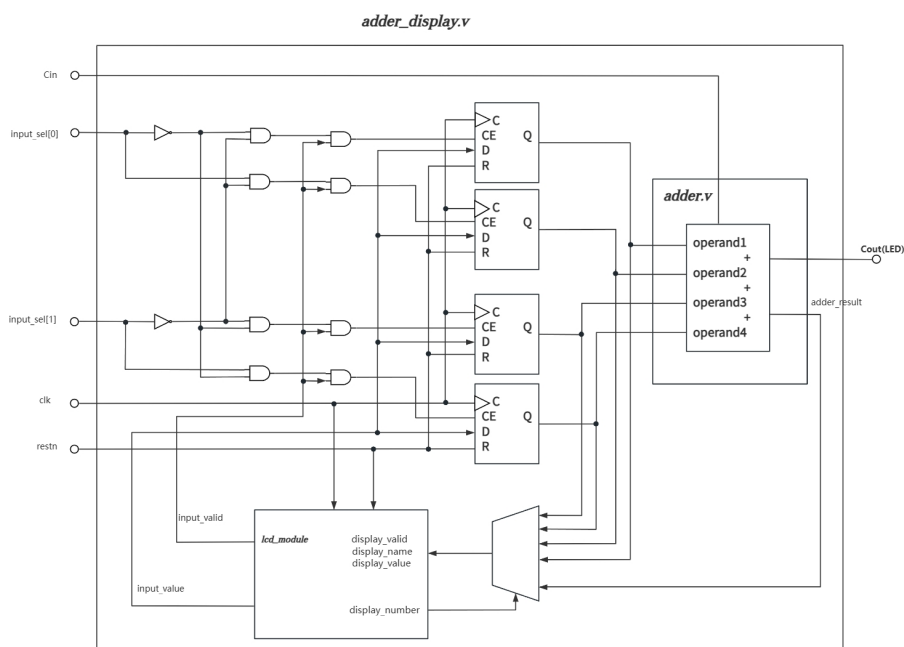


图 2: 实验原理图: 改进后的四元加法器

依照指导书的原理图进行改进，将 input_sel 改为两位，仿照 2-4 译码器控制输入状态。修改 adder 模块新增加数并连接对应的线路，其余逻辑与加法器设计类似。

四、实验过程

1. 基础加法器实现：2 个 32 位数相加

使用 Vivado 创建项目文件，将老师提供的项目代码导入（为防止项目配置问题，此处使用文本级的复制粘贴进行导入），依据指导书完成设计文件、仿真文件、约束文件各部分，检查代码无误后，

依次运行 Synthesis 与 Implementation, 最后生成位流文件 Bitstream; 代码项目生成完毕后, 连接实验箱, 将位流文件烧录至开发板上, 依照设计进行功能验证。



图 3: 实验运行结果

根据上图验证计算结果:

- $0x00111111 + 0x00033665 + 0 = 0x00144776$, 计算结果正确;
- $0x00002222 + 0x00033665 + 1 = 0x00035888$, 计算结果正确, cin 功能正常。

经测验, 加法器各功能正常符合预期设计, 项目配置与代码逻辑均准确无误。

2. 加法器功能改进: 4 个 32 位数相加

完成了基础的二元加法器后, 依据实验要求, 继续对功能模块进行改进, 使其可以实现四个 32 位数的加法计算。从两个数的运算改为四个数的运算, 变量增加, 阅读代码的实现逻辑可知, 从基础的模块设计到外层 LCD 屏相关的显示配置, 都需进行修改, testbench 测试文件与.xdc 约束文件也应进行相应的改动, 修改后的具体代码如下:

i. 设计文件

```
1 // 四元32位加法器
2 // adder32.v
3 `timescale 1ns / 1ps
4 module adder32(
5     input [31:0] operand1,
6     input [31:0] operand2,
7     input [31:0] operand3,
8     input [31:0] operand4,
9     input      cin,
10
```

```

11     output [31:0] result,
12     output [1:0]  cout
13 );
14 assign {cout,result} = operand1 + operand2 + operand3 + operand4 + cin;
15
16 endmodule

```

```

1 // 加法器LCD屏显示
2 // adder_display.v
3 module adder_display(
4     //时钟与复位信号
5     input clk,
6     input resetn,    //后缀"n"代表低电平有效
7     //拨码开关，用于选择输入数和产生cin
8     input [1:0] input_sel, //00:输入为加数1(add_operand1);01:输入为加数2(
9         add_operand2);以此类推.....
10    input sw_cin,
11    //led灯，用于显示cout
12    output [1:0] led_cout,
13
14    //触摸屏相关接口，不需要更改
15    output lcd_rst,
16    output lcd_cs,
17    output lcd_rs,
18    output lcd_wr,
19    output lcd_rd,
20    inout [15:0] lcd_data_io,
21    output lcd_bl_ctr,
22    inout ct_int,
23    inout ct_sda,
24    output ct_scl,
25    output ct_rstn
26 );
27 //-----{调用加法模块}begin
28 reg [31:0] adder_operand1;
29 reg [31:0] adder_operand2;
30 reg [31:0] adder_operand3;
31 reg [31:0] adder_operand4;
32 wire adder_cin;
33 wire [31:0] adder_result ;
34 wire [1:0] adder_cout;
35 adder32 adder_module(
36     .operand1(adder_operand1),
37     .operand2(adder_operand2),
38     .operand3(adder_operand3),
39     .operand4(adder_operand4),
40     .cin      (adder_cin      ),
41     .result   (adder_result  ),
42     .cout     (adder_cout     )

```

```

43     );
44     assign adder_cin = sw_cin;
45     assign led_cout  = adder_cout;
46 //-----{调用加法模块}end
47
48 //-----{调用触摸屏模块}begin-----//
49 //-----{实例化触摸屏}begin
50 //此小节不需要更改
51     reg          display_valid;
52     reg [39:0] display_name;
53     reg [31:0] display_value;
54     wire [5 :0] display_number;
55     wire          input_valid;
56     wire [31:0] input_value;
57
58     lcd_module lcd_module(
59         .clk          (clk          ),    //10Mhz
60         .resetn       (resetn       ),
61         //调用触摸屏的接口
62         .display_valid (display_valid ),
63         .display_name  (display_name  ),
64         .display_value (display_value ),
65         .display_number (display_number),
66         .input_valid   (input_valid   ),
67         .input_value   (input_value   ),
68         //lcd触摸屏相关接口，不需要更改
69         .lcd_rst       (lcd_rst       ),
70         .lcd_cs        (lcd_cs        ),
71         .lcd_rs        (lcd_rs        ),
72         .lcd_wr        (lcd_wr        ),
73         .lcd_rd        (lcd_rd        ),
74         .lcd_data_io   (lcd_data_io   ),
75         .lcd_bl_ctr    (lcd_bl_ctr    ),
76         .ct_int        (ct_int        ),
77         .ct_sda        (ct_sda        ),
78         .ct_scl        (ct_scl        ),
79         .ct_rstn       (ct_rstn       )
80     );
81 //-----{实例化触摸屏}end
82
83 //-----{从触摸屏获取输入}begin
84 //根据实际需要输入的数修改此小节，
85 //建议对每一个数的输入，编写单独一个always块
86 //当input_sel为00时，表示输入数为加数1，即operand1
87     always @(posedge clk)
88     begin
89         if (!resetn)
90         begin
91             adder_operand1 <= 32'd0;
92         end

```

```

93     else if (input_valid && !input_sel[1] && !input_sel[0])// 00
94     begin
95         adder_operand1 <= input_value;
96     end
97 end
98
99 //当input_sel为01时，表示输入数为加数2，即operand2
100 always @(posedge clk)
101 begin
102     if (!resetn)
103     begin
104         adder_operand2 <= 32'd0;
105     end
106     else if (input_valid && !input_sel[1] && input_sel[0])// 01
107     begin
108         adder_operand2 <= input_value;
109     end
110 end
111 //当input_sel为10时，表示输入数为加数3，即operand3
112 always @(posedge clk)
113 begin
114     if (!resetn)
115     begin
116         adder_operand3 <= 32'd0;
117     end
118     else if (input_valid && input_sel[1] && !input_sel[0])// 10
119     begin
120         adder_operand3 <= input_value;
121     end
122 end
123
124 //当input_sel为11时，表示输入数为加数4，即operand4
125 always @(posedge clk)
126 begin
127     if (!resetn)
128     begin
129         adder_operand4 <= 32'd0;
130     end
131     else if (input_valid && input_sel[1] && input_sel[0])// 11
132     begin
133         adder_operand4 <= input_value;
134     end
135 end
136 //-----{从触摸屏获取输入}end
137
138 //-----{输出到触摸屏显示}begin
139 //44块显示区域从1开始编号，编号为1~44，
140 always @(posedge clk)
141 begin
142     case(display_number)

```

```

143         6'd1 :
144         begin
145             display_valid <= 1'b1;
146             display_name  <= "ADD_1";
147             display_value <= adder_operand1;
148         end
149         6'd2 :
150         begin
151             display_valid <= 1'b1;
152             display_name  <= "ADD_2";
153             display_value <= adder_operand2;
154         end
155         6'd3 :
156         begin
157             display_valid <= 1'b1;
158             display_name  <= "ADD_3";
159             display_value <= adder_operand3;
160         end
161         6'd4 :
162         begin
163             display_valid <= 1'b1;
164             display_name  <= "ADD_4";
165             display_value <= adder_operand4;
166         end
167         6'd5 :
168         begin
169             display_valid <= 1'b1;
170             display_name  <= "RESUL";
171             display_value <= adder_result;
172         end
173         default :
174         begin
175             display_valid <= 1'b0;
176             display_name  <= 40'd0;
177             display_value <= 32'd0;
178         end
179     endcase
180 end
181 //-----{输出到触摸屏显示}end
182 //-----{调用触摸屏模块}end-----//
183 endmodule

```

添加输入变量 operand3 与 operand4，并将其配置到相应的 LCD 屏幕区域；将拨码开关扩至两位，使其可以通过四种状态控制当前要输入的变量；由于加数数量变成了四个，最终输出的 cout 需扩为两位，相应地，也将 LED 输出灯改为两位。文件设计完成，检查无误后，编写仿真文件，进行波形仿真验证其功能。

ii. 仿真文件

```

1 // 仿真验证
2 // testbench.v
3 `timescale 1ns / 1ps
4 module testbench;
5     // Inputs
6     reg [31:0] operand1;
7     reg [31:0] operand2;
8     reg [31:0] operand3;
9     reg [31:0] operand4;
10    reg      cin;
11    // Outputs
12    wire [31:0] result;
13    wire [1:0]  cout;
14    // Instantiate the Unit Under Test (UUT)
15    adder32 uut (
16        .operand1(operand1),
17        .operand2(operand2),
18        .operand3(operand3),
19        .operand4(operand4),
20        .cin(cin),
21        .result(result),
22        .cout(cout)
23    );
24    initial begin
25        // Initialize Inputs
26        operand1 = 0;
27        operand2 = 0;
28        operand3 = 0;
29        operand4 = 0;
30        cin = 0;
31        // Wait 100 ns for global reset to finish
32        #100;
33        // Add stimulus here
34    end
35    always #10 operand1 = $random; // $random为系统任务，产生一个随机的32位数
36    always #10 operand2 = $random; // #10 表示等待10个单位时间(10ns)，即每过10ns，
    赋值一个随机的32位数
37    always #10 operand3 = $random;
38    always #10 operand4 = $random;
39
40    always #10 cin = {$random} % 2; //加了拼接符，{$random}产生一个非负数，除2取余
    得到0或1
41 endmodule

```

验证波形数据

op1 = 0x603921C0	op2 = 0x13259F26	
op3 = 0xDB461AB6	op4 = 0x3E99837D	cin = 0x0
res = 0x8D3E5F19	cout = 0x1	

经计算，加法器功能正常，计算结果准确无误，进位输出 cout 符合预期。

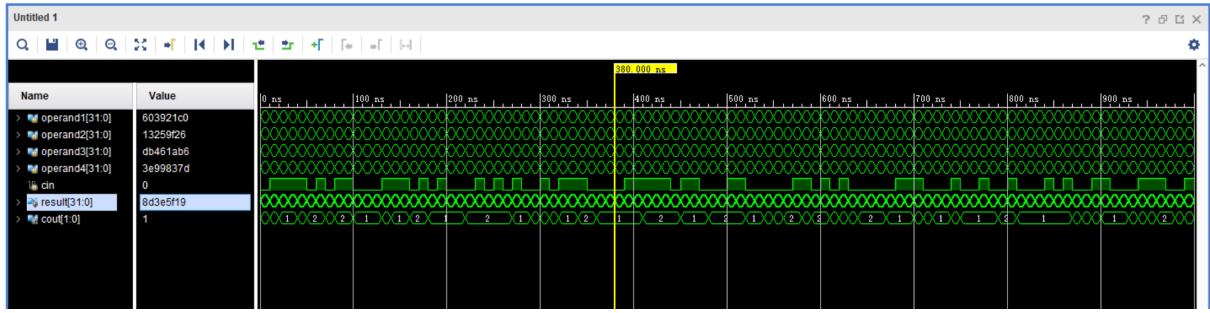


图 4: 仿真波形输出

代码确认无误后，根据引脚对应关系，修改约束文件。

iii. 约束文件

```

1 // 约束文件
2 // adder.xdc
3 set_property PACKAGE_PIN AC19 [get_ports clk]
4 set_property PACKAGE_PIN H7 [get_ports led_cout[0]]
5 set_property PACKAGE_PIN D5 [get_ports led_cout[1]]
6 set_property PACKAGE_PIN Y3 [get_ports resetn]
7 set_property PACKAGE_PIN AC21 [get_ports input_sel[0]]
8 set_property PACKAGE_PIN AD24 [get_ports input_sel[1]]
9 set_property PACKAGE_PIN AC22 [get_ports sw_cin]
10
11 set_property IOSTANDARD LVCMOS33 [get_ports clk]
12 set_property IOSTANDARD LVCMOS33 [get_ports led_cout[0]]
13 set_property IOSTANDARD LVCMOS33 [get_ports led_cout[1]]
14 set_property IOSTANDARD LVCMOS33 [get_ports resetn]
15 set_property IOSTANDARD LVCMOS33 [get_ports input_sel[0]]
16 set_property IOSTANDARD LVCMOS33 [get_ports input_sel[1]]
17 set_property IOSTANDARD LVCMOS33 [get_ports sw_cin]
18
19 #lcd
20 set_property PACKAGE_PIN J25 [get_ports lcd_rst]
21 set_property PACKAGE_PIN H18 [get_ports lcd_cs]
22 set_property PACKAGE_PIN K16 [get_ports lcd_rs]
23 set_property PACKAGE_PIN L8 [get_ports lcd_wr]
24 set_property PACKAGE_PIN K8 [get_ports lcd_rd]
25 set_property PACKAGE_PIN J15 [get_ports lcd_bl_ctr]
26 set_property PACKAGE_PIN H9 [get_ports {lcd_data_io[0]}]
27 set_property PACKAGE_PIN K17 [get_ports {lcd_data_io[1]}]
28 set_property PACKAGE_PIN J20 [get_ports {lcd_data_io[2]}]
29 set_property PACKAGE_PIN M17 [get_ports {lcd_data_io[3]}]
30 set_property PACKAGE_PIN L17 [get_ports {lcd_data_io[4]}]
31 set_property PACKAGE_PIN L18 [get_ports {lcd_data_io[5]}]
32 set_property PACKAGE_PIN L15 [get_ports {lcd_data_io[6]}]
33 set_property PACKAGE_PIN M15 [get_ports {lcd_data_io[7]}]
34 set_property PACKAGE_PIN M16 [get_ports {lcd_data_io[8]}]

```

```

35 set_property PACKAGE_PIN L14 [get_ports {lcd_data_io[9]}]
36 set_property PACKAGE_PIN M14 [get_ports {lcd_data_io[10]}]
37 set_property PACKAGE_PIN F22 [get_ports {lcd_data_io[11]}]
38 set_property PACKAGE_PIN G22 [get_ports {lcd_data_io[12]}]
39 set_property PACKAGE_PIN G21 [get_ports {lcd_data_io[13]}]
40 set_property PACKAGE_PIN H24 [get_ports {lcd_data_io[14]}]
41 set_property PACKAGE_PIN J16 [get_ports {lcd_data_io[15]}]
42 set_property PACKAGE_PIN L19 [get_ports ct_int]
43 set_property PACKAGE_PIN J24 [get_ports ct_sda]
44 set_property PACKAGE_PIN H21 [get_ports ct_scl]
45 set_property PACKAGE_PIN G24 [get_ports ct_rstn]
46
47 set_property IOSTANDARD LVCMOS33 [get_ports lcd_rst]
48 set_property IOSTANDARD LVCMOS33 [get_ports lcd_cs]
49 set_property IOSTANDARD LVCMOS33 [get_ports lcd_rs]
50 set_property IOSTANDARD LVCMOS33 [get_ports lcd_wr]
51 set_property IOSTANDARD LVCMOS33 [get_ports lcd_rd]
52 set_property IOSTANDARD LVCMOS33 [get_ports lcd_bl_ctr]
53 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[0]}]
54 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[1]}]
55 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[2]}]
56 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[3]}]
57 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[4]}]
58 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[5]}]
59 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[6]}]
60 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[7]}]
61 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[8]}]
62 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[9]}]
63 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[10]}]
64 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[11]}]
65 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[12]}]
66 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[13]}]
67 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[14]}]
68 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[15]}]
69 set_property IOSTANDARD LVCMOS33 [get_ports ct_int]
70 set_property IOSTANDARD LVCMOS33 [get_ports ct_sda]
71 set_property IOSTANDARD LVCMOS33 [get_ports ct_scl]
72 set_property IOSTANDARD LVCMOS33 [get_ports ct_rstn]

```

约束文件部分, 将拨码开关 AC21 与 AD24 配置为输入状态控制的 input_sel[1:0], 第三位 AC22 配置为进位输入 sw_cin; LED 灯新增一位 D5, 用于显示添加的进位输出。修改完引脚关系约束, 还需补全相应的 IO 电平约束。

代码完成后, 将生成的位流文件导入实验箱烧录, 对其进行功能验证, 得到以下结果:

根据上图验证计算结果:

- 加数: ADD_1 = 0x00000776 ADD_2 = 0x0000012D
 ADD_3 = 0x00000450 ADD_4 = 0x88776655 cin = 0

结果: RESUL = 0x88777348

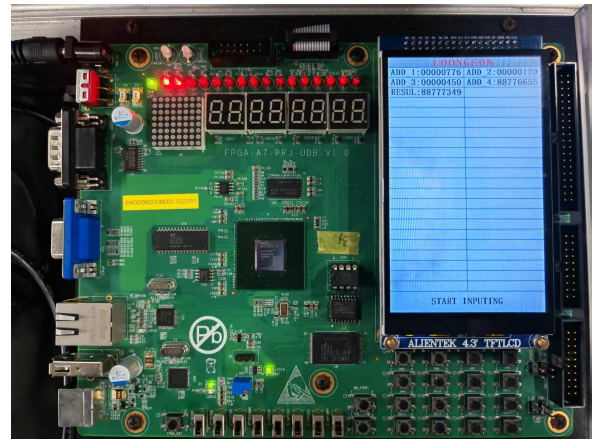
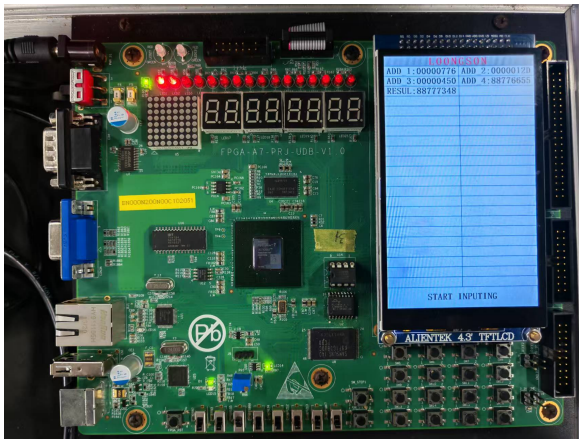


图 5: 计算运行结果

- 测试 cin，打开第三位拨码开关（对应 sw_cin），RESUL 变为 0x88777349
经计算，输出结果正确，进位输入功能正常。

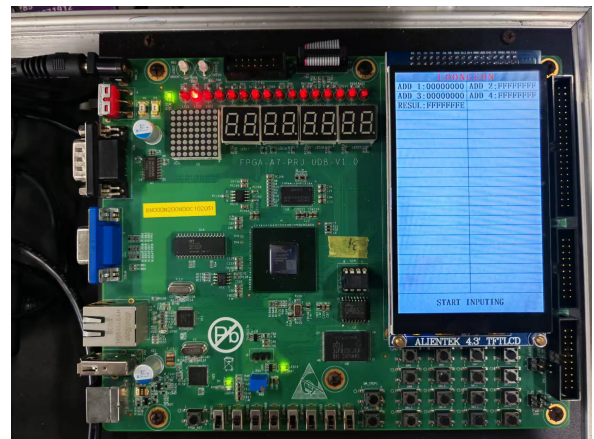
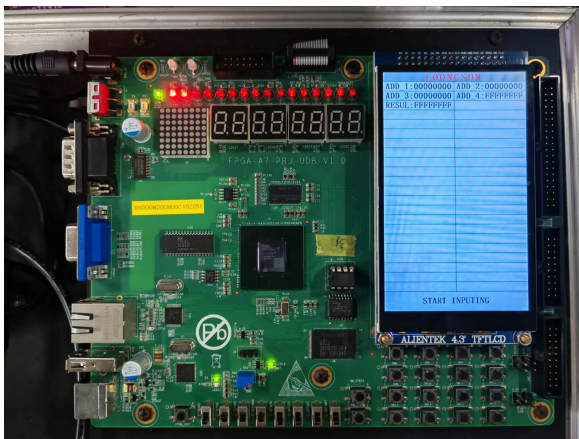


图 6: LED 功能测试

计算 $0xFFFFFFFF + 0xFFFFFFFF$ 发生进位，此时观察到左侧一位 LED 灯熄灭；
进位输出 LED 灯功能正常，符合实验预期。

五、总结感想

本次实验中，首先通过老师提供的源码实现了两个 32 位数的加法器模块，在此基础上经过修改，将其改进为四个 32 位数的加法器。依照实验要求，准确完成了本次实验。

通过这次实验，我学会了如何使用 Vivado 完成硬件项目的设计，在过程中学习理解硬件模块设计流程。同时，还对加法运算的基本原理有了更加深入的认知。对于 Vivado 的三类基本文件——设计文件、约束文件、仿真文件有了较为全面的了解。本次实验也让我体会到硬件设计中时序与约束的重要性，只有规范完成引脚与电平约束，才能保证电路正常稳定工作。今后我会继续加强对硬件描述语言和 FPGA 开发流程的练习，为后续更复杂的数字系统设计打下坚实基础。